

Data Structure Assignment

Submission Instructions (Mandatory)

Before starting the tasks, carefully read the following instructions:

1. GitHub Repository Creation

- Create a public GitHub repository for your assignment.
- Use the following naming convention:
DS/SEP-2025/RegistrationNumber
- **Example:**
DS/SEP-2025/151
- **Important:** Since GitHub does not allow slashes (/) in repository names, replace all slashes with hyphens (-) when creating your repo.
Example Repo Name on GitHub:
DS-SEP-2025-151

2. Submission Form

- A submission form will be enabled on **15th September 2025**.
- You must submit the **GitHub repository link** through this form.
- **README.md** must include

3. Repository Visibility

- Ensure your **GitHub repository is public**.
- **Private repositories will not be considered for evaluation.**

Tasks

Task 1: Implement a Stack and Queue

Objective:

Understand basic linear data structures and implement them from scratch.

Task:

- Implement **Stack** using arrays and linked lists.
- Implement **Queue** using arrays and linked lists.
- Include the following operations:
 - **Push, Pop, Peek, IsEmpty** (for Stack)
 - **Enqueue, Dequeue, Front, IsEmpty** (for Queue)

Skills to Demonstrate:

Data Structure Assignment

- Understanding of **arrays and linked lists**
- Implementation of **LIFO (Stack)** and **FIFO (Queue)** principles

Task 2: Binary Search Tree (BST) Operations

Objective:

Learn about hierarchical data structures and recursive algorithms.

Task:

- Create a **Binary Search Tree**.
- Implement the following:
 - **Insertion**
 - **Search**
 - **In-order, Pre-order, Post-order Traversals**
- Bonus: Implement **Deletion** in BST.

Skills to Demonstrate:

- Understanding of **trees and recursion**
- Ability to handle **dynamic data structures**

Task 3: Sorting and Searching Algorithms

Objective:

Apply classic sorting and searching techniques to datasets.

Task:

- Implement the following algorithms:
 - **Sorting:** Bubble Sort, Merge Sort, Quick Sort
 - **Searching:** Binary Search, Linear Search
- Compare their **time complexities** and show execution time for large inputs.

Skills to Demonstrate:

- Algorithmic thinking
- Complexity analysis (**Big-O notation**)
- Practical implementation and performance measurement